

ALFA PDF Reader

Guia de melhorias e atualizacoes futuras

Introducao

Este documento e um guia de melhorias profissionais para futuras atualizacoes do ALFA PDF Reader. Esta dividido em 4 niveis de prioridade (P0 a P3), ordenados por impacto e esforco. Use como roteiro de desenvolvimento para evoluir o app de um leitor de PDF basico para uma solucao profissional de nivel comercial.

P0 - Critico (Essencial para profissionalismo)

[P0-1] Code Signing do Instalador

Atualmente o instalador nao e assinado digitalmente, causando avisos do Windows SmartScreen ("Editor desconhecido"). Para um app profissional, isso destrui a confianca do usuario. Solucao: adquirir um certificado de codigo (OV ou EV) e configurar electron-builder.

```
// package.json build
"win": {
  "certificateFile": "build/cert.pfx",
  "certificatePassword": process.env.CERT_PASSWORD
}
```

- Custo aproximado: USD 100-300/ano (OV), USD 300-700/ano (EV).
- EV removing o aviso do SmartScreen mais rapidamente.
- Alternativa gratuita: SignPath Foundation oferece code signing gratuito para projetos open-source.

[P0-2] Ativacao do ASAR

O build atual usa asar: false, expondo todo o codigo fonte em texto plano na pasta do app. Isso e inefficiente, inseguro e gera warnings do electron-builder. Ativar ASAR empacota o app em um arquivo unico, melhora performance de leitura e protege o codigo.

```
"asar": true,
"asarUnpack": [
  "node_modules/pdfjs-dist/**",
  "node_modules/react-pdf/**"
]
```

- asarUnpack necessario para modulos com workers/binarios externos.
- Reduz tempo de inicializacao e protege propriedade intelectual.

[P0-3] Tratamento Robusto de Erros

O app precisa de tratamento de erros centralizado em tres frentes:

- Main: try/catch em todos os ipcMain.handle com log estruturado em arquivo.
- Renderer: ErrorBoundary React para capturar erros de UI sem crashar o app.
- Renderer: toast de erro amigavel em portugues para operacoes que falham.
- Rotacao de logs (eleitos: logs em %APPDATA%/ALFA-PDF-Reader/logs/).

P1 - Alta Prioridade (Diferencial competitivo)

[P1-1] Historico de Arquivos Recentes

Memorizar os ultimos 10 PDFs abertos, persistir em arquivo JSON em `app.getPath("userData")` e mostrar na `WelcomeScreen`. Click rapido para reabrir.

- Adicionar hook `useRecentFiles` com `electron-store` ou `fs` manual.
- Persistir: `path`, `name`, `lastOpened` (timestamp), `thumbnail` (opcional).
- Limitar a 10 itens, remover se arquivo nao existir mais.

[P1-2] Paginacao e Navegacao Avancada

- Mini-mapa (thumbnails lateral) das paginas com scroll synchronized.
- Indice/Navegacao por bookmarks (TOC) — `pdfjs` suporta `getOutline()`.
- Busca de texto em todas as paginas (`pdfjs.getTextContent()`).
- Highlights persistentes (marcacoes em posicoes do texto).
- Modo de leitura continua (scroll, nao paginado).

[P1-3] Anotacoes e Marcacoes

- Sublinhado e marca-texto em trechos de texto.
- Notas adesivas (sticky notes) em posicoes do PDF.
- Desenho livre (caneta) — util para assinaturas digitais.
- Exportar PDF com anotacoes usando `pdf-lib`.

[P1-4] Suporte a Multiplas Linguas (i18n)

- Adicionar `i18next` + `react-i18next`.
- Extrair todos os textos hard-coded em português para template JSON.
- Suporte inicial: `pt-BR` (nativo) e `en-US` (internacionalizacao).
- Detector automatico de idioma do sistema (`app.getLocale()`).

P2 - Media Prioridade (Refinamento profissional)

[P2-1] Temas Personalizados

- Alm do dark/light atual, suporte a 3-4 temas preset (Crimson, Midnight, Sepia).
- Temas ja tem infraestrutura (ThemeContext) — basta adicionar presets.
- Configuracao persistida em userData.

[P2-2] Atalhos de Teclado Padronizados

- Implementar suporte completo a aceleradores globais.
- Ctrl+O abrir, Ctrl+W fechar tab, Ctrl+P imprimir, Ctrl+S salvar.
- Ctrl+Tab/Ctrl+Shift+Tab navegar entre tabs.
- F11 tela cheia, Ctrl+= zoom in, Ctrl+- zoom out, Ctrl+0 reset.
- Pagina anterior/proximo: PageUp/PageDown (ja existe parcial).

[P2-3] Performance e Otimizacao de Memoria

- Virtualizacao de paginas: renderizar so visiveis (react-virtual).
- Cache de thumbnails em disco (nao reload toda vez).
- Limitar PDFs simultaneos abertos (atualmente sem limite).
- Lazy load do react-pdf e pdfjs só quando necessario.
- Memoizacao agressiva de componentes com React.memo + useMemo.

[P2-4] Drag-and-Drop

- Arrastar PDFs do Explorer diretamente para dentro do app.
- onDrop no container principal + pedir arquivo via IPC.
- Feedback visual durante o hover (borda destacada).

P3 - Baixa Prioridade (Features de nicho)

[P3-1] Integracao com Cloud

- Abrir PDFs diretamente de Google Drive, OneDrive, Dropbox.
- OAuth flow com firebase/auth ou manual.
- Suporte a URLs http(s):// para abrir PDFs remotos.

[P3-2] Conversao de PDF

- PDF para imagem (PNG/JPG) usando pdfjs + canvas.
- PDF para texto puro (.txt).
- Mesclar multiplos PDFs (pdf-lib ja esta no projeto).
- Dividir PDF em paginas separadas.
- Comprimir/reduzir tamanho do PDF.

[P3-3] Telemetria Anonima (Opcional)

- Coletar metricas de uso: paginas abertas, tempo de leitura, crashes.
- USAR opt-in explicito do usuario (checkbox em settings).
- Backend gratis: PostHogCloud ou Mixpanel free tier.
- Nunca coletar conteudo do PDF, apenas metadata de uso.

[P3-4] CI/CD Automation

- GitHub Actions para build automatico em push/tag.
- Build paralelo em Windows, macOS e Linux.
- Auto-publicar release no GitHub quando tag v* e PUSHADA.
- Executar typecheck + lint + testes em cada PR antes de merge.

Roadmap Sugerido (por versao)

v2.1.0 - Estabilidade e Profissionalismo

Foco: corrigir as pendencias tecnicas que impedem o app de ser considerado "serio". Recomendado antes de qualquer feature nova.

- [P0-1] Code Signing do instalador.
- [P0-2] Ativacao do ASAR.
- [P0-3] Tratamento robusto de erros + ErrorBoundary.
- [P1-4] Internacionalizacao (pt-BR + en-US).
- [P2-1] Temas personalizados (mais 2-3 presets).

v2.2.0 - Experiencia de Leitura

Foco: transformar o app em um leitor de classe premium.

- [P1-1] Arquivos recentes.
- [P1-2] Indice (bookmarks TOC).
- [P1-2] Busca de texto em todas as paginas.
- [P1-2] Mini-mapa lateral de thumbnails.
- [P2-3] Virtualizacao de paginas (performance).
- [P2-4] Drag-and-drop de arquivos.

v2.3.0 - Anotacoes

Foco: ferramentas de marcacao, transformando em solucao de productividade.

- [P1-3] Marca-texto e sublinhado.
- [P1-3] Notas adesivas.
- [P1-3] Desenho livre (caneta).
- [P1-3] Exportar PDF com anotacoes.
- [P2-2] Atalhos de teclado completos.

v2.4.0 - Multiplataforma e Conversao

Foco: alcancar mais usuarios e criar utilidades.

- [P3-4] CI/CD via GitHub Actions.
- [P3-2] Conversao PDF <-> imagem / texto.
- [P3-2] Mesclar/dividir PDFs.
- [P3-1] Abrir PDFs de URLs remotas.

v3.0.0 - Cloud e Premium

Foco: expansao para servico, fans e telemetria.

- [P3-1] Integracao com Google Drive, OneDrive, Dropbox.
- [P3-3] Telemetria opt-in.
- Sincronizacao de historico/anotacoes entre dispositivos.
- Versao macOS nativa com notarizacao.

Checklist de Qualidade Antes de Cada Release

Antes de publicar qualquer versao, validar todos os pontos abaixo:

Build

- npm run typecheck passou sem erros.
- npm run lint passou sem warnings.
- npm test passou todos os testes.
- npm run build gerou pastas out/main, out/preload e out/renderer.
- npm run dist gerou instalador em release/ sem erros.
- latest.yml presente em release/.

Funcional

- Teste manual: abrir PDF, navegar paginas, zoom in/out.
- Teste manual: imprimir (silencioso e nativo).
- Teste manual: salvar como PDF.
- Teste manual: abrir PDF via duplo-clique no Explorer.
- Teste manual: abrir PDF via WhatsApp / arquivo recebido.
- Teste de auto-update: instalar v anterior e validar upgrade.

Versionamento

- Atualizar "version" no package.json.
- Atualizar "uninstallDisplayName" no nsis se mudou major.
- Criar tag v*.*.* no git.
- Criar GitHub Release correspondente e fazer upload de:
 - ALFA PDF Reader Setup X.X.X.exe
 - latest.yml
 - latest.yml.sha256 (opcional).